UNIVERSIDAD DE LA SALLE
ESCUELA DE CIENCIAS BÁSICAS
PROGRAMA DE CIENCIA DE DATOS

Manual de Programación y Preubas - Museo

Presentado por:
Sebastián Gutiérrez Guayacundo

Docente:
RODRIGO ARANDA FERNANDEZ

Lenguaje de Programación II
Semestre III – 2026
Bogotá, Colombia

Contenido

1. Definición del problema real 3
2. Algoritmo en Python (Lógica de Solución) 3
3. Código en Visual Studio Code 7
4. Prueba de Escritorio (Caso: Compra de Smart TV) 9
Resultado: 9

1. Definición del problema real

El Museo requiere automatizar la preservación de su patrimonio artístico. El reto principal es el monitoreo del ciclo de vida de las obras, las cuales están sujetas a degradación natural y requieren mantenimiento preventivo obligatorio cada 5 años. Además, el sistema debe gestionar la disponibilidad compartida (préstamos internacionales), asegurando que si una obra no está en sala (por restauración o cesión), las solicitudes de otros museos queden registradas en una lista de espera. Finalmente, la dirección requiere una valoración patrimonial actualizada y en tiempo real de todos los activos (cuadros y esculturas).

2. Algoritmo en Python (Lógica de Solución)

El sistema se basa en una arquitectura de **Jerarquía de Clases** y validación temporal:

1. **Carga de Catálogo:** Instanciación de objetos especializados (Cuadro o Escultura) que heredan de una base común.
2. **Seguridad:** Validación de roles mediante la clase Usuario para permitir el acceso a módulos específicos.
3. **Evaluación de Mantenimiento:** Un algoritmo compara la fecha actual con la ultima_restauracion. Si la diferencia supera los 1,825 días, el estado cambia automáticamente.
4. **Gestión de Disponibilidad:** Uso de condicionales para verificar si una obra está Expuesta antes de permitir una cesión.
5. **Cálculo de Activos:** Iteración sobre la lista de inventario para sumar el atributo valor de cada objeto.
6. **Reporte de Estados:** Generación de un resumen final que incluye la visualización de la lista de espera mediante tipos opcionales (Optional).

main_museo.py

```
from modelos_museo import ObraDeArte, Cuadro, Escultura, Usuario
from datetime import datetime, timedelta
from typing import List

def ejecutar_sistema_completo():
    # --- 1. MÓDULO DE SEGURIDAD ---
    print("="*50)
    print("     SISTEMA INTEGRADO MUSEO LA SALLE")
    print("="*50)

    admin = Usuario("sebas_admin", "lasalle2026", "Encargado")
    print(f"[*] Autenticando usuario: {admin.username}...")

    if not admin.autenticar("sebas_admin", "lasalle2026"):
        print("[ERROR] Acceso denegado.")
        return

    print("[OK] Acceso concedido. Cargando módulos...\n")
```

```
    # --- 2. CARGA DE CATÁLOGO (Encargado) ---
    obra1 = Cuadro("La Noche Estrellada", "Van Gogh", "Postimpresionismo", 100000000, "1889",
"Impresionista", "Óleo")
    obra2 = Escultura("El Pensador", "Rodin", "Moderna", 50000000, "1904", "Realismo",
"Bronce")

    inventario: List[ObraDeArte] = [obra1, obra2]
    print(f"[CATÁLOGO] Se han registrado {len(inventario)} obras con éxito.")

    # --- 3. PROCESO DE RESTAURACIÓN (Restaurador Jefe) ---
    print("\n" + "-"*30)
    print("EJECUTANDO CHEQUEO DE MANTENIMIENTO")
    print("-"*30)

    # Simulamos que 'El Pensador' no se ha tocado en 6 años
    obra2.ultima_restauracion = datetime.now() - timedelta(days=6*365)

    for obra in inventario:
        # Chequeo automático (Requisito: Proceso diario)
        obra.chequear_mantenimiento_automatico()

        # Simulación de daño accidental (Requisito: Envío inmediato)
        if "Noche Estrellada" in obra.titulo:
            print(f"[!] Alerta: Se detectó humedad en '{obra.titulo}'")
            obra.enviar_a_restauracion("Daño por Humedad")

    # --- 4. GESTIÓN DE CESIONES Y VALORACIÓN (Director) ---
    print("\n" + "-"*30)
    print("MÓDULO DE DIRECCIÓN")
    print("-"*30)

    # Requisito: Suma total
    total = sum(o.valor for o in inventario)
    print(f"[REPORTE] Valoración total del patrimonio: ${total}")

    # Requisito: Cesiones y lista de espera
    print("\n[GESTIÓN DE PRÉSTAMOS]")
    obra1.ceder_a_museo("Museo del Louvre", 25000000)
    obra1.ceder_a_museo("Museo del Prado", 18000000) # Debería quedar en espera

    # --- 5. REPORTE FINAL DE ESTADOS ---
    print("\n" + "="*50)
    print("      RESUMEN FINAL DE OPERACIONES")
    print("="*50)
    for obra in inventario:
        estado_espera = f" | En espera para: {obra.museo_en_espera}" if obra.museo_en_espera else
""
        print(f"Obra: {obra.titulo:20} | Estado: {obra.estado:12}{estado_espera}")
```

```
if __name__ == "__main__":
    ejecutar_sistema_completo()
```

modelos_museo.py

```
from abc import ABC, abstractmethod
from datetime import datetime, timedelta
from typing import List, Optional

class Restauracion:
    def __init__(self, tipo: str, fecha_inicio: datetime):
        self.tipo = tipo
        self.fecha_inicio = fecha_inicio
        self.fecha_fin: Optional[datetime] = None

    def __str__(self) -> str:
        estado = f"Finalizada el {self.fecha_fin.date()}" if self.fecha_fin else "En curso"
        return f"[{self.fecha_inicio.date()}] Tipo: {self.tipo} - {estado}"

class ObraDeArte(ABC):
    def __init__(self, titulo: str, autor: str, periodo: str, valor: float, fecha_creacion: str):
        self.titulo = titulo
        self.autor = autor
        self.periodo = periodo
        self.valor = valor
        self.fecha_creacion = fecha_creacion
        self.fecha_entrada_museo = datetime.now()
        self.estado = "Expuesta"
        self.ultima_restauracion: datetime = self.fecha_entrada_museo
        self.historial_restauraciones: List[Restauracion] = []
        self.museo_en_espera: Optional[str] = None
        self.importe_cesion: float = 0.0
        self.museo_cesionario: Optional[str] = None

    def enviar_a_restauracion(self, tipo: str):
        if self.estado != "Restauración":
            nueva_res = Restauracion(tipo, datetime.now())
            self.historial_restauraciones.append(nueva_res)
            self.estado = "Restauración"
            print(f"[MANTENIMIENTO] {self.titulo} enviada a restauración ({tipo}).")

    # Para que Pylance no se queje de timedelta:
    def chequear_mantenimiento_automatico(self):
        cinco_anos = timedelta(days=5*365)
        if datetime.now() - self.ultima_restauracion >= cinco_anos:
            self.enviar_a_restauracion("Mantenimiento Quinquenal Automático")

    def finalizar_restauracion(self):
        if self.estado == "Restauración":
            self.historial_restauraciones[-1].fecha_fin = datetime.now()
```

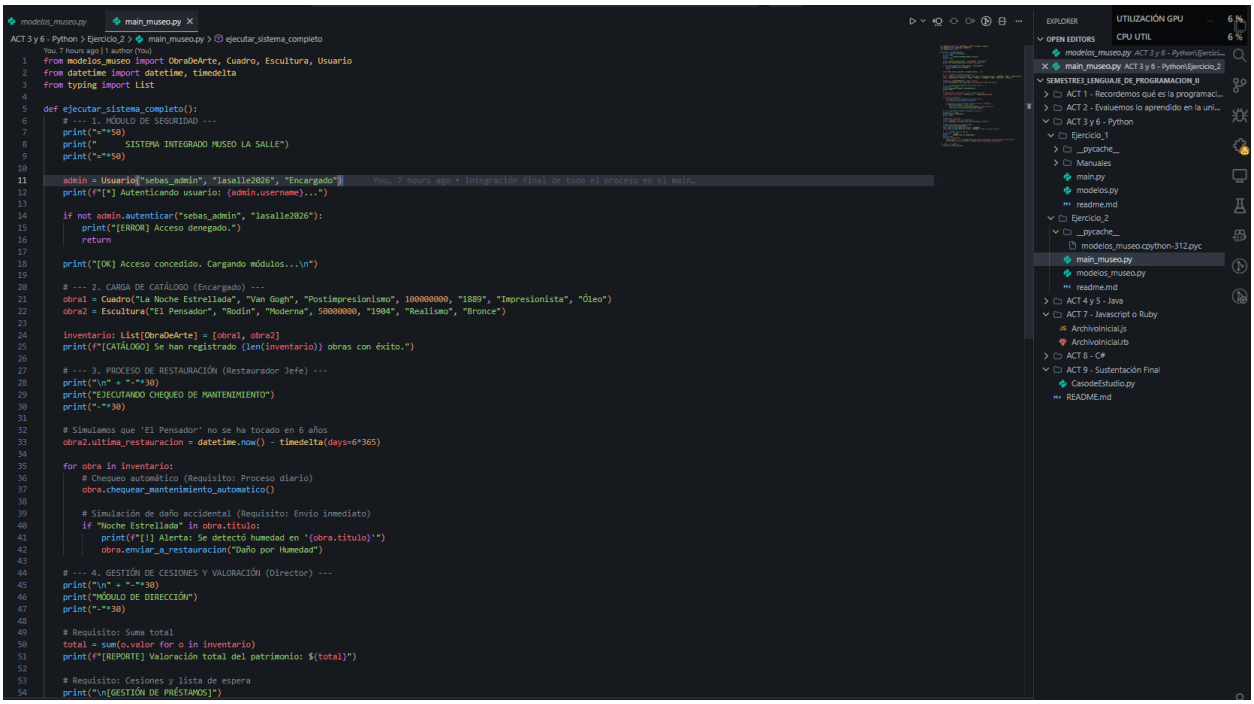
```
        self.estado = "Expuesta"
        self.ultima_restauracion = datetime.now()
        print(f"[SOPORTE] Restauración de {self.titulo} finalizada.")

    def ceder_a_museo(self, nombre_museo: str, importe: float):
        if self.estado == "Expuesta":
            self.estado = "Cedida"
            self.museo_cesionario = nombre_museo
            self.importe_cesion = importe
            print(f"[DIRECTOR] Obra '{self.titulo}' cedida a {nombre_museo} por ${importe}")
        else:
            self.museo_en_espera = nombre_museo
            print(f"[DIRECTOR] {nombre_museo} en espera. '{self.titulo}' está actualmente
{self.estado}")

    @abstractmethod
    def mostrar_detalle(self) -> str:
        pass

    def __str__(self) -> str:
        return f"[{self.__class__.__name__}] {self.titulo} - {self.autor} ($${self.valor})"

class Cuadro(ObraDeArte):
    def __init__(self, titulo: str, autor: str, periodo: str, valor: float, fecha_creacion: str, estilo: str,
tecnica: str):
        super().__init__(titulo, autor, periodo, valor, fecha_creacion)
        self.estilo = estilo
        self.tecnica = tecnica

    def mostrar_detalle(self) -> str:
        return f"Estilo: {self.estilo}, Técnica: {self.tecnica}"

class Escultura(ObraDeArte):
    def __init__(self, titulo: str, autor: str, periodo: str, valor: float, fecha_creacion: str, estilo: str,
material: str):
        super().__init__(titulo, autor, periodo, valor, fecha_creacion)
        self.estilo = estilo
        self.material = material

    def mostrar_detalle(self) -> str:
        return f"Estilo: {self.estilo}, Material: {self.material}"

class Usuario:
    def __init__(self, username: str, password: str, rol: str):
        self.username = username
        self.password = password
        self.rol = rol # Director, Restaurador, Encargado

    def autenticar(self, user: str, psw: str) -> bool:
        return self.username == user and self.password == psw
```

3. Código en Visual Studio Code

main_museo.py

Este archivo funciona como el controlador. Su función es:

1. **Controlador de Flujo:** Simula las tareas diarias de un museo: autenticar personal, verificar el estado físico de las piezas y gestionar solicitudes de directores internacionales.
2. **Lógica Temporal:** Utiliza la librería datetime para simular el paso de los años y validar la eficacia de los disparadores automáticos de mantenimiento.



```python
from modelos_museo import ObraDeArte, Cuadro, Escultura, Usuario
from datetime import datetime, timedelta
from typing import List

def ejecutar_sistema_completo():
    # --- 1. MÓDULO DE SEGURIDAD ---
    print("="*50)
    print("      SISTEMA INTEGRADO MUSEO LA SALLE")
    print("="*50)

    admin = Usuario("sebas_admin", "lasalle2026", "Encargado")
    print(f"[*] Autenticando usuario: {admin.username}...")

    if not admin.autenticar("sebas_admin", "lasalle2026"):
        print("[ERROR] Acceso denegado.")
        return

    print("[OK] Acceso concedido. Cargando módulos...\n")

    # --- 2. CARGA DE CATÁLOGO (Encargado) ---
    obra1 = Cuadro("La Noche Estrellada", "Van Gogh", "Postimpresionismo", 100000000, "1889", "Impresionista", "Óleo")
    obra2 = Escultura("El Pensador", "Rodin", "Moderna", 50000000, "1904", "Realismo", "Bronce")

    inventario: List[ObraDeArte] = [obra1, obra2]
    print(f"[CATÁLOGO] Se han registrado {len(inventario)} obras con éxito.")

    # --- 3. PROCESO DE RESTAURACIÓN (Restaurador Jefe) ---
    print("\n" + "-"*30)
    print("EJECUTANDO CHEQUEO DE MANTENIMIENTO")
    print("-"*30)

    # Simulamos que 'El Pensador' no se ha tocado en 6 años
    obra2.ultima_restauracion = datetime.now() - timedelta(days=6*365)

    for obra in inventario:
        # Chequeo automático (Requisito: Proceso diario)
        obra.chequear_mantenimiento_automatico()

        # Simulación de daño accidental (Requisito: Envío inmediato)
        if "Noche Estrellada" in obra.titulo:
            print(f"[!] Alerta: Se detectó humedad en '{obra.titulo}'")
            obra.enviar_a_restauracion("Daño por Humedad")

    # --- 4. GESTIÓN DE CESIONES Y VALORACIÓN (Director) ---
    print("\n" + "-"*30)
    print("MÓDULO DE DIRECCIÓN")
    print("-"*30)

    # Requisito: Suma total
    total = sum(o.valor for o in inventario)
    print(f"[REPORTE] Valoración total del patrimonio: ${total}")

    # Requisito: Cesiones y lista de espera
    print("\n[GESTIÓN DE PRÉSTAMOS]")
```

modelos_museo.py

Este archivo es el núcleo del sistema y aplica los pilares de la POO:

- **Herencia y Especialización:** Se utiliza ObraDeArte como clase base. Las subclases Cuadro y Escultura heredan atributos comunes, pero manejan datos específicos (técnica vs material).
- **Polimorfismo:** El método abstracto mostrar_detalle obliga a cada subclase a definir cómo se presenta su información técnica, permitiendo un manejo uniforme en el inventario.
- **Composición:** La clase ObraDeArte contiene una lista de objetos Restauracion, permitiendo llevar un historial clínico detallado de cada pieza.

modelos_museo.py × main_museo.py

ACT 3 y 6 - Python > Ejercicio_2 > modelos_museo.py > ObraDeArte > __init__

You, 7 hours ago | 1 author (You)

```python
from abc import ABC, abstractmethod
from datetime import datetime, timedelta
from typing import List, Optional

You, 8 hours ago | 1 author (You)
class Restauracion:
    def __init__(self, tipo: str, fecha_inicio: datetime):
        self.tipo = tipo
        self.fecha_inicio = fecha_inicio
        self.fecha_fin: Optional[datetime] = None

    def __str__(self) -> str:
        estado = f"Finalizada el {self.fecha_fin.date()}" if self.fecha_fin else "En curso"
        return f"[{self.fecha_inicio.date()}] Tipo: {self.tipo} - {estado}"

You, 7 hours ago | 1 author (You)
class ObraDeArte(ABC):
    def __init__(self, titulo: str, autor: str, periodo: str, valor: float, fecha_creacion: str):
        self.titulo = titulo
        self.autor = autor
        self.periodo = periodo
        self.valor = valor  You, 8 hours ago • Implementar lógica de mantenimiento quinquenal …
        self.fecha_creacion = fecha_creacion
        self.fecha_entrada_museo = datetime.now()
        self.estado = "Expuesta"
        self.ultima_restauracion: datetime = self.fecha_entrada_museo
        self.historial_restauraciones: List[Restauracion] = []
        self.museo_en_espera: Optional[str] = None
        self.importe_cesion: float = 0.0
        self.museo_cesionario: Optional[str] = None

    def enviar_a_restauracion(self, tipo: str):
        if self.estado != "Restauración":
            nueva_res = Restauracion(tipo, datetime.now())
            self.historial_restauraciones.append(nueva_res)
            self.estado = "Restauración"
            print(f"[MANTENIMIENTO] {self.titulo} enviada a restauración ({tipo}).")

    # Para que Pylance no se queje de timedelta:
    def chequear_mantenimiento_automatico(self):
        cinco_anos = timedelta(days=5*365)
        if datetime.now() - self.ultima_restauracion >= cinco_anos:
            self.enviar_a_restauracion("Mantenimiento Quinquenal Automático")

    def finalizar_restauracion(self):
        if self.estado == "Restauración":
            self.historial_restauraciones[-1].fecha_fin = datetime.now()
            self.estado = "Expuesta"
            self.ultima_restauracion = datetime.now()
            print(f"[SOPORTE] Restauración de {self.titulo} finalizada.")

    def ceder_a_museo(self, nombre_museo: str, importe: float):
        if self.estado == "Expuesta":
            self.estado = "Cedida"
```

modelos_museo.py × main_museo.py

ACT 3 y 6 - Python > Ejercicio_2 > modelos_museo.py > ObraDeArte > __init__

```python
class ObraDeArte(ABC):

    def ceder_a_museo(self, nombre_museo: str, importe: float):
        if self.estado == "Expuesta":
            self.estado = "Cedida"
            self.museo_cesionario = nombre_museo
            self.importe_cesion = importe
            print(f"[DIRECTOR] Obra '{self.titulo}' cedida a {nombre_museo} por ${importe}")
        else:
            self.museo_en_espera = nombre_museo
            print(f"[DIRECTOR] {nombre_museo} en espera. '{self.titulo}' está actualmente {self.estado}")

    @abstractmethod
    def mostrar_detalle(self) -> str:
        pass

    def __str__(self) -> str:
        return f"[{self.__class__.__name__}] {self.titulo} - {self.autor} ($ {self.valor})"

You, 8 hours ago | 1 author (You)
class Cuadro(ObraDeArte):
    def __init__(self, titulo: str, autor: str, periodo: str, valor: float, fecha_creacion: str, estilo: str, tecnica: str):
        super().__init__(titulo, autor, periodo, valor, fecha_creacion)
        self.estilo = estilo
        self.tecnica = tecnica

    def mostrar_detalle(self) -> str:
        return f"Estilo: {self.estilo}, Técnica: {self.tecnica}"

You, 7 hours ago | 1 author (You)
class Escultura(ObraDeArte):
    def __init__(self, titulo: str, autor: str, periodo: str, valor: float, fecha_creacion: str, estilo: str, material: str):
        super().__init__(titulo, autor, periodo, valor, fecha_creacion)
        self.estilo = estilo
        self.material = material

    def mostrar_detalle(self) -> str:
        return f"Estilo: {self.estilo}, Material: {self.material}"

You, 7 hours ago | 1 author (You)
class Usuario:
    def __init__(self, username: str, password: str, rol: str):
        self.username = username
        self.password = password
        self.rol = rol # Director, Restaurador, Encargado

    def autenticar(self, user: str, psw: str) -> bool:
        return self.username == user and self.password == psw
```

4. Prueba de Escritorio Caso: Gestión de Obras de Arte)

Paso	Acción Realizada	Variable: obra.estado	Variable: museo_en_espera	Salida en Consola (Log)
1	Login Exitoso	Expuesta	None	[OK] Acceso concedido
2	Alerta de Humedad	Restauración	None	[!] Alerta: Se detectó humedad...
3	Solicitud Louvre	Restauración	Museo del Louvre	[DIRECTOR] Louvre en espera...
4	Solicitud Prado	Restauración	Museo del Prado	[DIRECTOR] Prado en espera...

Resultado:

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS    GITLENS

sebas@SoyySebas MINGW64 ~/Semestre3_Lenguaje_de_Programacion_II (main)
$ C:/Users/sebas/AppData/Local/Programs/Python/Python312/python.exe "c:/Users/sebas/Semestre3_Lenguaje_de_Programacion_II/ACT 3 y 6 - Python/Ejercicio_2/main_museo.py"
==================================================
      SISTEMA INTEGRADO MUSEO LA SALLE
==================================================
[*] Autenticando usuario: sebas_admin...
[OK] Acceso concedido. Cargando módulos...

[CATÁLOGO] Se han registrado 2 obras con éxito.

------------------------------
EJECUTANDO CHEQUEO DE MANTENIMIENTO
------------------------------
[!] Alerta: Se detectó humedad en 'La Noche Estrellada'
[MANTENIMIENTO] La Noche Estrellada enviada a restauración (Daño por Humedad).
[MANTENIMIENTO] El Pensador enviada a restauración (Mantenimiento Quinquenal Automático).

------------------------------
MÓDULO DE DIRECCIÓN
------------------------------
[REPORTE] Valoración total del patrimonio: $150000000

[GESTIÓN DE PRÉSTAMOS]
[DIRECTOR] Museo del Louvre en espera. 'La Noche Estrellada' está actualmente Restauración
[DIRECTOR] Museo del Prado en espera. 'La Noche Estrellada' está actualmente Restauración

==================================================
      RESUMEN FINAL DE OPERACIONES
==================================================
Obra: La Noche Estrellada  | Estado: Restauración | En espera para: Museo del Prado
Obra: El Pensador          | Estado: Restauración

sebas@SoyySebas MINGW64 ~/Semestre3_Lenguaje_de_Programacion_II (main)
$
```

Tras ejecutar la simulación del Museo La Salle, se obtienen los siguientes resultados:

- **Detección de Anomalías y Automatización**: El sistema demostró su capacidad de respuesta inmediata ante riesgos físicos (humedad detectada en "La Noche Estrellada") y riesgos temporales (mantenimiento automático quinquenal en "El Pensador"). Esto confirma que el algoritmo de prevención de daños funciona correctamente.

- **Integridad de la Colección:** La valoración total de $150,000,000 calculada por el módulo de dirección confirma que la herencia permite consolidar datos financieros de objetos heterogéneos (Cuadros y Esculturas) de manera precisa.

- **Gestión de Disponibilidad Crítica:** El log de préstamos evidencia que el sistema es capaz de "negar" una cesión si la obra no está en condiciones óptimas para el traslado. La asignación del Museo del Prado a la lista de espera garantiza que ninguna obra sea comprometida mientras está en proceso de restauración.

- **Seguridad y Auditoría:** La trazabilidad de los mensajes permite al usuario sebas_admin supervisar cada cambio de estado, cumpliendo con los estándares de control requeridos para una institución de alto nivel como la Universidad de La Salle.